

Guía de trabajo en grupo

Producto de la Unidad 1 — Matriz de decisión tecnológica · Bases de Datos III

Cada sección trae un recordatorio corto y un ejemplo, si algo no les quedó claro en la explicación en vivo, esto les debería alcanzar para seguir solos.

Recordatorio rápido: familias y motores

MongoDB — documental: documentos tipo JSON sin esquema fijo. Sirve cuando los datos varían mucho entre registros (catálogos).

Redis — clave-valor: todo en memoria RAM, una clave que apunta a un valor. Sirve cuando la velocidad es crítica (cachés, sesiones).

Neo4j — grafos: nodos y relaciones guardadas como conexiones directas. Sirve para redes y recomendaciones.

Cassandra — columnar: datos distribuidos en muchos servidores. Sirve para volúmenes de escritura enormes (sensores, logs).

PostgreSQL — relacional (SQL): tablas fijas con filas y columnas. Sirve cuando la consistencia es innegociable (pagos, transacciones).

Nombres de integrantes del grupo:

Emanuel Gil Restrepo

Maria Paula Cañarte

Yeisson Alexander Rico Ñustes

Gabriela Morales Bedoya

Fecha: 20/09/2026

1. Caso real

Plantilla: "[Sistema] que maneja [tipo de datos] para [quién lo usa]". Tiene que ser específico, no sirve "una empresa" o "una app".

Boletería y gestión de eventos culturales

Una plataforma para publicar eventos (conciertos, obras, ferias) y vender boletos.

Motor / familia	Qué guardaría ahí
Documental (MongoDB)	Catálogo de eventos — cada tipo de evento tiene campos distintos (concierto, teatro, feria).
Clave-valor (Redis)	Disponibilidad de boletos en tiempo real (evitar sobreventa), carrito de compra temporal.
Grafos (Neo4j)	Recomendación de eventos según amigos que van a asistir o intereses compartidos.
SQL (fuera de alcance)	Pagos y facturación de boletos.

2. Tabla comparativa de motores candidatos

Elijan UNA parte de su sistema (no todo a la vez). Para esa parte, elijan 2-3 motores candidatos y califiquen cada criterio como Alta/Media/Baja, con una justificación de una línea.

¿Qué pregunta cada criterio?

Consistencia — ¿qué tan grave es que el dato esté desactualizado por un instante? (grave, como en dinero → SQL; no grave, como un contador de likes → NoSQL puede.)

Latencia — ¿qué tan rápido necesito la respuesta: milisegundos, o puede tardar un poco más?

Escalabilidad — ¿Cuánto va a crecer esto: cientos, miles, millones de registros?

Costo operativo — ¿cuánto cuesta mantener y operar ese motor a ese volumen?

Criterio	Motor 1: MongoDB	Motor 2: Redis	Motor 3: Cassandra
Consistencia	Media – Es flexible con los datos que no cambian frecuente	Media – Puede perder datos si se cae el servidor	Media – Busca estar siempre disponible
Latencia	Media – Buena para lecturas frecuentes de eventos	Alta – Respuesta en milisegundos	Alta – Responde rápido
Escalabilidad	Alta – Puede crecer fácilmente	Media – Limitada por la RAM	Alta – Soporta muchos datos entre servidores

Criterio	Motor 1: MongoDB	Motor 2: Redis	Motor 3: Cassandra
Costo operativo	Medio – Costo razonable para un catálogo	Alto – Guardar todo en RAM es más caro	Medio – Es más complejo

Ejemplo de referencia (catálogo de canciones de una app de streaming) — la suya no tiene que verse igual, es solo para que vean cómo se llena:

Criterio	MongoDB (documental)	PostgreSQL (SQL)	Cassandra (columnar)
Consistencia	Baja — eventual, aceptable en un catálogo	Alta — más de lo que este caso exige	Media — eventual, ajustable
Latencia	Media — buena para lecturas frecuentes	Media-alta — JOINS con metadatos variables cuestan	Baja — pensada para escritura masiva
Costo operativo	Medio	Medio — paga funciones que aquí no se usan	Alto — más nodos para tolerancia a fallos

3. Justificación final

Motor 1 — MongoDB:

Para el catálogo de eventos, elegimos MongoDB porque la variedad y flexibilidad de los datos es la prioridad, evidenciado en que los conciertos, obras de teatro y ferias tienen campos diferentes y no todos necesitan la misma información.

Motor 2 — Redis:

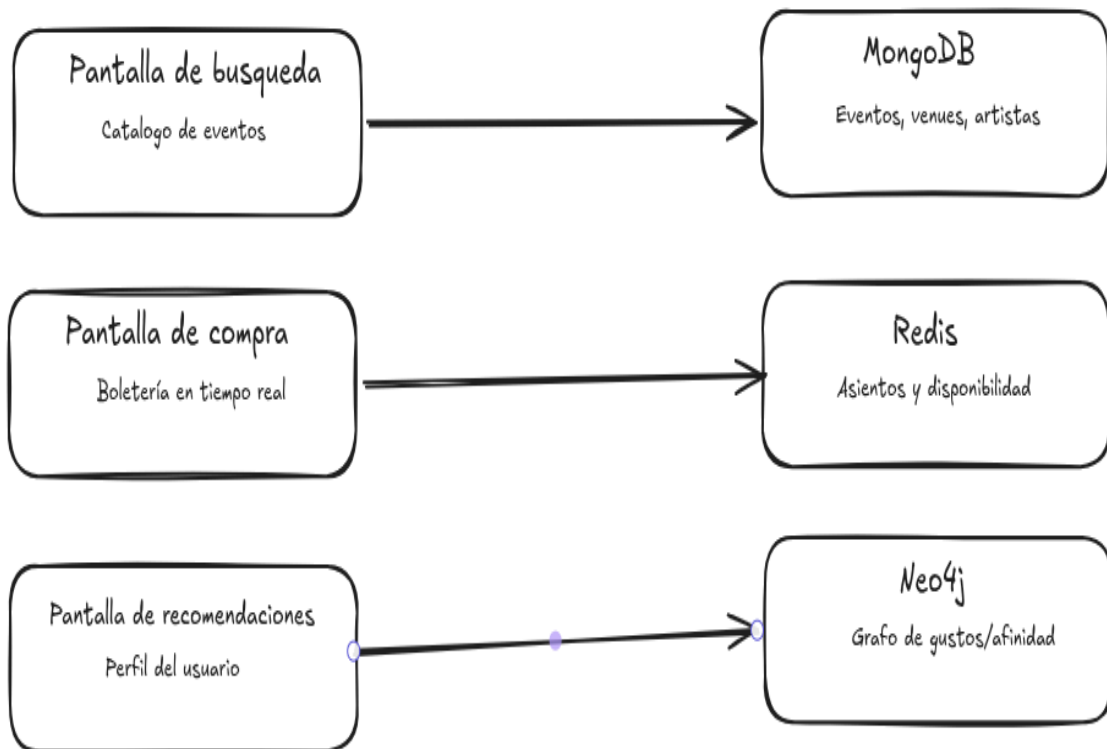
Para la disponibilidad y compra temporal de boletos, elegimos Redis porque la velocidad y actualización en tiempo real es la prioridad, evidenciado en que se necesita controlar la disponibilidad de boletos inmediatamente para evitar la sobreventa y mantener los carritos de compra por un tiempo limitado.

Motor 3 — Neo4j:

Para la recomendación de eventos, elegimos Neo4j porque las relaciones entre usuarios, amigos, intereses y eventos es la prioridad, evidenciado en que la plataforma debe recomendar eventos según los amigos que asistirán o los intereses que los usuarios comparten.

4. Arquitectura inicial

*Pregunta guía: si tu aplicación tuviera 3 pantallas principales, ¿qué pantalla llama a qué base de datos?
No necesitan UML formal, un diagrama de bloques simple basta.*



5. Repositorio Git

¿Por qué ahora, aunque no haya código? Ese primer commit ya cuenta como evidencia de trabajo desde el día uno — en la Unidad 8 van a necesitar el historial completo.

- x Repositorio creado en GitHub
- x git clone — traje el repositorio a mi computador
- x git add . — marqué los archivos para guardar
- x git commit -m "..." — guardé los cambios con un mensaje
- x git push — subí los cambios a GitHub

URL del repositorio: <https://github.com/yarico17-crypto/boleteria-eventos-bd3>
